Reproducing the Temeraire Redis Results on WSL2 and Bare-Metal Linux

Aldo Costa
aldo.costa@hhu.de
Heinrich-Heine-Universität
Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire is a hugepage-aware extension of TCMalloc that aims to improve application performance by preserving hugepage coverage and reducing TLB pressure. The original OSDI 2021 paper evaluates Temeraire through application studies, a Google fleet experiment, and a production rollout. This report deliberately narrows reproduction to the part that can be approximated with public code: the Redis 6.0.9 case study.

The artifact evolved from a generic allocator benchmark into a Redis-focused reproduction by pinning Redis, selecting a historical public TCMalloc revision, building separate legacy and hugepage-aware shared libraries, and checking the loaded allocator at runtime. Reaching a trustworthy measurement took two environments and two series. Docker/WSL2 drifted by more than 200 kRPS while the allocator was held constant, and the first bare-metal series recorded no CPU time even though the paper normalizes its throughput for CPU.

The corrected series reproduces the paper's release-on result. Across four balanced pairs the Temeraire delta is +0.401% in raw throughput and +0.474% in requests per CPU second. All four pairs are positive, and both 95% intervals exclude zero while containing the paper's +0.44%. Release-off centres on zero and stays unresolved across two independent samples. Normalization proves to be worth under a tenth of a percentage point here, because a single-threaded Redis under a saturating benchmark holds one core throughout, and the release rate the paper omits matters more than the metric: at 64 MiB/s every pair favors the legacy pageheap. The artifact, the per-trial data behind every number here, and the source of this report are public at `https://github.com/Eyecatch3r/a-hugepage-aware-memory-allocator`.

1 Introduction

Memory allocation is often treated as a local implementation detail of C and C++ programs. Temeraire starts from a different observation: allocator decisions shape where objects land in memory, whether huge pages remain usable, and how much address-translation work the processor must perform later. Hunter et al. therefore connect allocator design to fleet efficiency as well as the direct cost of `malloc` and `free` [1].

A full reproduction of the Temeraire paper is outside the scope of this seminar artifact. The paper's main evidence comes from Google's production environment, including internal workloads, large-scale telemetry, and fleet-level rollout data. Those conditions are not available externally. This report therefore focuses on the Redis 6.0.9 case study: a public workload where the paper gives enough benchmark detail to support a local reproduction attempt.

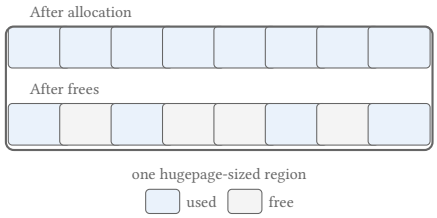

Figure 1: Fragmentation inside a hugepage-sized region. Free memory exists, but live objects prevent the whole region from being released intact. Redrawn after Figure 1 in Hunter et al. [1].

The question here is narrower than rebuilding Google's internal allocator binary: can the Redis comparison be approximated with public code closely enough to observe the same small-effect regime? Reaching that point required two environments. Docker/WSL2 provided the development and diagnostic stage; a dedicated Linux node provided the final native measurements.

Artifact availability. The artifact is public at `https://github.com/Eyecatch3r/a-hugepage-aware-memory-allocator`. The repository holds the setup and benchmark scripts, the per-trial data of every run, the audit and aggregation tools, the unit tests, a static results explorer, and the source of this document. A clone regenerates every reported figure without a benchmark run, and `README.md` states the commands. Section 5 reports the numbers that those commands reproduce.

2 Background and Temeraire Mechanism

Processors translate virtual addresses to physical addresses through page tables. Because page-table walks are expensive, recent translations are cached in translation lookaside buffers (TLBs). With ordinary 4 KiB pages, large working sets can require many translations. A 2 MiB hugepage covers the same address range as 512 ordinary pages, so one hugepage TLB entry can cover much more memory than one ordinary-page entry.

The gain comes from needing fewer cached translations, not from loading 2 MiB of data into cache at once. Whether that gain is realized depends on layout: hugepage mappings require suitably aligned and contiguous regions. Linux Transparent Huge Pages (THP) can create such mappings, but its success depends partly on how the user-space allocator arranges live objects.

Figure 1 shows the central trade-off. Returning small free pieces to the OS can reduce resident memory, but it may split hugepage mappings. Keeping the hugepage intact preserves TLB coverage,

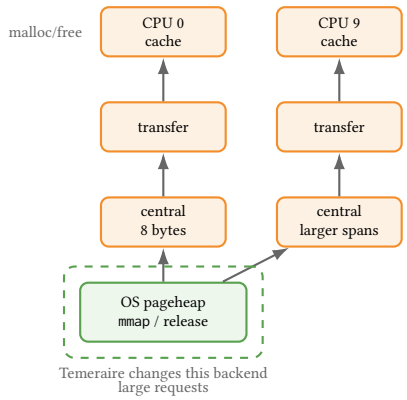

Figure 2: Simplified TCMalloc hierarchy. Temeraire changes the backend pageheap layer while leaving the upper cache structure intact. Redrawn after Figure 3 in Hunter et al. [1].

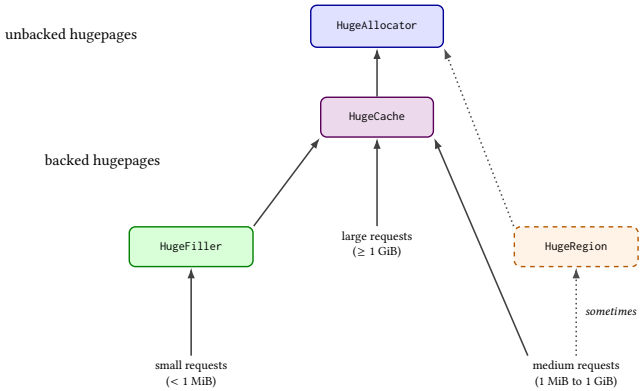

Figure 3: Temeraire's component structure. Small requests are routed to `HugeFiller`, large requests to `HugeCache`, and medium requests usually to `HugeCache` but occasionally to `HugeRegion`. Adapted after Figure 6 in Hunter et al. [1].

but leaves some physical memory idle. Temeraire exists to make this trade-off explicit inside the allocator backend.

TCMalloc is organized as a hierarchy of caches. Many small allocations are served by upper per-CPU, transfer, or central-cache layers. Larger page-granularity requests eventually reach the pageheap, which obtains memory from the OS and releases unused spans. Temeraire modifies this backend pageheap layer rather than replacing the whole allocator.

This localization sets the shape of the reproduction. The experiment compares two backend configurations of one allocator, the legacy pageheap and the hugepage-aware path, rather than two separate allocators.

Figure 3 summarizes the backend components relevant for the artifact. `HugeAllocator` manages hugepage-aligned virtual address ranges. `HugeCache` keeps completely empty backed hugepages for reuse. `HugeFiller` handles small allocations and is the key component for Redis. `HugeRegion` handles medium-sized allocations that would otherwise create large slack regions.

The key Redis-relevant policy is in `HugeFiller`. For a request of size K, it prefers a hugepage whose longest free run is just large enough for the request. When several candidates are equivalent, it prefers the fuller one. This avoids consuming long free runs unnecessarily and avoids disturbing nearly empty hugepages that might otherwise drain and be released.

The Redis list workload in the paper performs small list-node and string-value allocations. These do not reach `HugeFiller` individually. As Figure 2 shows, the per-CPU and central caches serve them; only when those caches need fresh backing spans does a page-level request arrive at the pageheap. `HugeFiller` places those spans, and the legacy pageheap places them without a hugepage-aware objective. Section 4 turns that difference into the measured hypothesis.

Periodic memory release complicates the picture. When release is enabled, TCMalloc can return memory to the OS using mechanisms such as `MADV_DONTNEED`. Release interacts with hugepage coverage and adds variance, so release-off carries the cleaner placement signal of the two conditions.

3 Reproduction Target and Artifact Evolution

The original paper reports application studies, a fleet experiment, and a production rollout. Only the Redis case study is realistic for a public seminar reproduction. The paper gives Redis 6.0.9, a legacy TCMalloc pageheap baseline, 2000 `redis-benchmark` trials, 1,000,000 requests per trial, and a workload combining a five-element push with a five-element read [1]. It also states the CPU family and LLVM commit used for compilation. The exact Linux distribution, kernel version, and THP policy for the Redis run are left unstated, so this report documents them locally instead of matching them.

The repository did not initially match this target. It was a useful allocator benchmark scaffold, but its defaults used Redis 7.2.5, compared glibc against open-source gperftools TCMalloc, and did not expose a clean legacy-pageheap versus Temeraire comparison. The first reproduction step was to narrow and correct the claim: the artifact would reproduce the Redis case-study methodology only, not the paper's full evaluation.

The paper's internal allocator build is unavailable, so the artifact uses a historical public `google/tcmalloc` commit, `8e534f50 7074`, which still exposes the `want_no_hpaa` mechanism needed to disable the hugepage-aware path. This makes a public legacy-versus-Temeraire split possible, but it should not be read as byte-identical to Google's internal artifact.

The public repository provides no ready-made shared libraries for Redis preloading. The setup script therefore adds a small wrapper target inside the cloned TCMalloc tree and builds two loadable variants. The Temeraire build links against `//tcmalloc`; the legacy build also links against `//tcmalloc:want_no_hpaa`. The benchmark harness selects the desired library through `LD_PRELOAD`.

Bazel was helpful because it let the artifact use TCMalloc's native build graph instead of reconstructing allocator dependencies, compiler flags, and link semantics in a separate build system. It also kept the comparison narrow: both shared libraries are produced from the same wrapper source, and the intended difference is the allocator target plus the extra `//tcmalloc:want_no_hpaa` dependency in

the legacy build. The cost was integration friction. Bazelisk had to be installed and pinned, the exact LLVM compiler path had to be passed through Bazel's action and repository environments, an external wrapper workspace did not inherit the right dependencies, and target visibility forced the wrapper into `tcmalloc/testing`.

Runtime verification is necessary. Redis reports `mem_allocator:libc` from its own build metadata even when `LD_PRELOAD` has replaced the process allocator, so the artifact checks `/proc/$pid/maps` to confirm that the intended shared library is mapped into the Redis process.

The final artifact reflects several failed or revised approaches. A modern `google/tcmalloc` checkout was not a convenient starting point for this wrapper strategy, and an external Bazel wrapper workspace did not naturally inherit all TCMalloc dependencies such as Abseil. The successful path was to add the wrapper inside the cloned TCMalloc source tree. Visibility was another obstacle: the `want_no_hpaa` target could not be linked from an arbitrary external package, so the wrapper had to be placed where TCMalloc's own visibility rules permitted the dependency.

A later validation run uncovered a symbol mismatch. The wrapper initially referenced background-release methods that were not present in the pinned historical TCMalloc revision. The shared objects built, but failed to load before Redis started. The fix was to resolve those methods dynamically when available, allowing the wrapper to load against the historical revision while preserving the intended behavior for newer revisions.

Compiler matching also proved more expensive than it first looked. The paper mentions LLVM commit `cd442157cf` and `-O3`. Matching that statement required a local LLVM/clang toolchain, then using it for Redis, gperftools, and the Bazel-based TCMalloc wrapper build. Later metadata confirmed use of a local LLVM 13.0.0 build from the longer `cd442157cf` commit. One older manifest still marked the exact LLVM path as disabled, so the compiler metadata is treated as authoritative.

The most important methodological difficulty was THP. Initial long runs under WSL2/Docker with THP in `madvise` mode recorded `AnonHugePages: 0 kB` for Redis. These runs looked like controlled allocator comparisons while leaving the hugepage mechanism itself unevidenced. The harness was then extended to record THP settings and periodic `smaps_rollup` snapshots, and the final paper-closer runs set THP to `always` before benchmarking. Even after that correction, later release-on runs showed large changes in absolute host throughput. That drift is what moved the same experiment to bare-metal Linux instead of leaving the WSL2 measurements as the final result.

4 Methodology

4.1 Common benchmark path

The repository retains a short direct Redis path for smoke tests and allocator preload checks. Measurements reported here use the paper-closer path. It fixes Redis to version 6.0.9 and uses 2000 trials, 50 clients, and pipeline depth 16. Each trial flushes the database and then runs two separate `redis-benchmark` invocations of 1,000,000 requests each:

- `LPUSH benchmark:list v1 v2 v3 v4 v5`;
- `LRANGE benchmark:list 0 4`.

This structure is one reading of an underspecified sentence, and its consequences should be stated plainly. The paper specifies 2000 trials, each trial making one million requests "to push 5 elements and read those 5 elements" [1]. Read literally, one request covers both the push and the read, so a trial performs one million of each. The local trial instead issues one million pushes and then one million reads, which doubles the command count to two million. It also separates the phases: because all pushes precede all reads, `LRANGE 0 4` returns the same final five elements on every read while the list holds five million, rather than returning each group just after that group was pushed. An interleaved implementation would keep the live set small and touch freshly written memory, which is a different access pattern and a different live-heap shape for the allocator under test. This report does not claim its choice is the paper's; it records the choice so that the comparison can be judged and repeated.

The runner stores the raw output of every operation as well as summary files, Redis logs, memory snapshots, allocator order, software revisions, and system metadata. Allocator order alternates between pairs. This does not remove drift between two consecutive blocks, but it avoids assigning the same allocator to the later position every time.

The primary metric is throughput in requests per second from `redis-benchmark`. LPUSH and LRANGE are collapsed into a combined push-plus-read rate using the harmonic mean

$$\frac{2}{1/r_{\mathrm{LPUSH}} + 1/r_{\mathrm{LRANGE}}}.$$

This treats the two throughputs as rates for sequential operations and reduces each block to one number, as the paper's single Redis delta per release mode does. The harmonic-mean aggregation itself is not specified in the paper. It is a local reconstruction choice because our public `redis-benchmark` harness records LPUSH and LRANGE as separate operation means, while the paper reports one Redis throughput delta for the combined push-five/read-five workload.

This metric is not the paper's metric, and the difference bounds every comparison in Section 5. The caption of Table 1 in Hunter et al. states that throughput there is "normalized for CPU", and the surrounding text gives the unit as requests per second per core [1]. The local figure is unnormalized: it is the request rate `redis-benchmark` reports, divided by nothing. A ratio of unnormalized rates cannot separate a genuine throughput gain from the same throughput bought with more CPU, which is exactly the distinction the paper's normalization preserves and the distinction its fleet-efficiency argument rests on. The local aggregation across two operations compounds this, since the paper reports no per-operation rates to compare against.

The consequence is stated once here and assumed afterwards. Where this report places a local value near a published one, that is numerical proximity between two differently constructed quantities. It is not agreement between two instances of one measurement, and no local number in this report can confirm or contradict the paper's normalized figures.

The hypothesis is modest, and follows from Section 2. If the placement heuristic in `HugeFiller` improves hugepage coverage for the spans backing Redis's small allocations, Redis should show a

small positive throughput delta against the legacy pageheap. The effect should be clearest with periodic release disabled, since release-on adds interaction with background memory return and can mask the placement signal.

4.2 Docker/WSL2 development stage

The first complete setup used a Debian 12 container on Docker Desktop and its shared WSL2 kernel. It was where the pinned builds, preload checks, benchmark parameters, THP instrumentation, and balanced ordering were developed. Once THP was set to `always`, Redis reported non-zero hugepage use. The container stayed a good development environment and made a poor measuring instrument, for reasons Section efsec:earlier-series gives.

Two faults found in that stage shaped everything after it. The runner started TCMalloc's background-actions thread without setting a release rate, which the pinned revision defaults to zero, so nothing labelled release-on had performed periodic release. The label had been wrong for the whole series. Fixing it exposed a second fault: the wrapper's dynamic symbol lookups had been failing silently, and `LD\_PRELOAD` had leaked from the Redis server to `redis-cli` and `redis-benchmark`, mixing client and server allocators into the same comparison. The wrapper now calls the linked TCMalloc API directly, the preload is scoped to the server, and every release-on run must log its requested rate before the benchmark proceeds.

Neither fix was the end of it. Rebuilding the wrapper needed one further Bazel dependency, and the container scripts still carried Windows line endings, which Linux reports as a missing `bash\r` interpreter rather than as a line-ending problem. Both cost more time than the allocator change they were meant to enable, which is the ordinary shape of this work: the substantive edit is small and the environment around it is not.

4.3 Bare-metal Linux stage

The WSL2 drift motivated a second workflow rather than a change to the existing container scripts. The native scripts preserve the same pinned source revisions, allocator wrappers, Redis workload, result layout, and balanced pair structure. They run directly on the dedicated node described in Table 1.

Table 1: Recorded bare-metal environment on node85.

Field	Recorded value
OS	Debian GNU/Linux 13 (trixie)
Kernel	`6.12.95+deb13-amd64`
Processor	Intel Xeon Gold 5318N
Topology	24 cores, 48 threads, one NUMA node
Memory	188 GiB
Virtualization	none detected
THP	`always`; defrag `madvise`

The move exposed a different set of practical problems. The shared home directory was too slow for Bazel's many small files, so the build and runs were moved to `/var/tmp`. The compute node could not fetch the pinned sources, which meant staging source and dependency archives for an offline build. The old LLVM revision also needed `-include cstdint` to build with Debian 13's host compiler. Finally, the German locale produced decimal commas in CSV files; the native runner now sets `LC_ALL=C`.

Three two-trial smoke runs and two ten-trial pilots came first. Four full balanced pairs then ran at 16 MiB/s, each pair covering legacy and Temeraire with release disabled and enabled. A second series kept release enabled and added four order-alternated pairs at 64 MiB/s and four at 256 MiB/s. All full allocator blocks contain 2000 trials, and each release-on Redis log confirms the requested rate. The archive holds 64,000 trial rows for the main four pairs and another 64,000 for the two added rates. A standard-library audit script re-derives every block mean from the raw trial files, checks trial identifiers, request counts, file counts, memory samples, THP state, release-rate confirmations, clean shutdowns, allocator order, and system metadata, and fails on any mismatch. Both halves of the archive pass.

5 Results

5.1 Why these are the second series of measurements

The first bare-metal series was complete and audited, and it was measuring the wrong thing in ways that only became visible afterwards. Three problems surfaced, in an order worth recording, because the last two were faults in the harness that produced those very numbers.

The first came from reading the paper again with the harness open alongside it. The caption of Table 1 in Hunter et al. says that its throughput is "normalized for CPU", and the surrounding text gives the unit as requests per second per core [1]. The local figure was an unnormalized request rate. A ratio of unnormalized rates cannot distinguish Temeraire serving more requests from Temeraire serving the same requests at higher CPU cost, and the artifact recorded no CPU time at all, so the distinction could not be recovered from the archived runs. The same reading showed that the paper's trial, one million requests "to push 5 elements and read those 5 elements", supports a second implementation in which one request performs both operations, and that the paper's Redis servers used Skylake Xeons while node85 carries an Ice Lake part. Two of those three gaps could be closed by changing the harness. The processor could not.

The second problem appeared eighteen hours into the first corrected attempt. A block died at trial 1875 of 2000 with `Connection reset by peer`, and the Redis log for that block showed a clean shutdown, which made the failure look like a transient fault on a shared node. It was not. The run had been started with a plain background `\&` from an interactive shell, so it stayed inside the SSH session, and the session had ended. Redis ignores `SIGHUP` at startup and `redis-benchmark` does not, so the hangup killed the client, left the server running, and the script's exit trap then shut that server down politely. The clean shutdown was the consequence of the failure rather than evidence against it.

The third problem destroyed a whole day of compute and was entirely self inflicted. `redis-benchmark` builds its CSV test name from the command line, so a run driven by an `EVAL` script carries the whole Lua source in that field, commas included. The harness read the rate by splitting the line on commas and taking the second field, which for such a run is the fragment `KEYS[1]`. Every trial of the first sixteen-hour block recorded that string, and the block mean came out as zero. Plain `LPUSH` and `LRANGE` names contain

no commas, which is why the fault stayed hidden until the second workload ran for the first time.

That last one was recoverable, because the runner writes each benchmark invocation's output to its own file before deriving anything from it. Those files held the correct rates the whole time, and a separate utility re-derives `trials.csv` and `summary.csv` from them, so the sixteen hours were repaired rather than repeated. The episode still points at a real weakness in the audit: it compares the summary against the aggregated trial table, and both descend from the same extraction, so it caught this only because `KEYS[1]` is not a number. A plausible but wrong value would have passed.

This section records the faults that changed a result or a method. The artifact also carries a longer protocol note that logs every dead end in order, including the ones that cost time without changing a number: the modern TCMalloc checkout that would not wrap, the external Bazel workspace that inherited no Abseil dependency, the target visibility that forced the wrapper into `tcmalloc/testing`, and the LLVM bootstrap that needed `-include cstdint` to build under a newer host compiler.

Four changes came out of the problems above. Each block now records the CPU time of the Redis server from `redis-cli info cpu`, which is that block's total because Redis restarts for every block. The workload is selectable, so both readings of the paper's trial sentence can be measured instead of one being chosen and defended. A failed benchmark invocation is retried, and every attempt reaches the audit as a warning so a retried block cannot pass as a clean one. Runs start in their own session through `setsid`, which removes the hangup described above.

5.2 Reported result

The corrected series ran for 90.7 hours on node85 and produced eight balanced pairs: four using the sequential workload of Section 4, and four using the combined `EVAL` reading. Every block holds 2000 trials, every release-on log confirms the 16 MiB/s rate, no block needed a retry, and the two launcher status files record a zero exit code. Of the 32 blocks, 28 re-derive byte-identical summaries from their raw per-trial files; the four that differ are the block set damaged by the extraction fault, repaired as described above.

Table 2: Reported result. Correction run on node85, sequential workload. Deltas in raw throughput and in requests per CPU second, the unit the paper reports. Intervals are four-pair Student-t intervals on the log-transformed pair ratios and accompany the mean.

Pair	Order	Release off	Release on
5	L first	−0.476%	+0.242%
6	T first	+0.156%	+0.555%
7	L first	−0.083%	+0.380%
8	T first	+0.490%	+0.426%
Mean		+0.022%	+0.401%
Median		+0.037%	+0.403%
95% interval		[−0.62, +0.67]	[+0.20, +0.61]
Mean per CPU second		+0.091%	+0.474%
95% interval		[−0.74, +0.93]	[+0.30, +0.65]
Paper		+0.75%	+0.44%

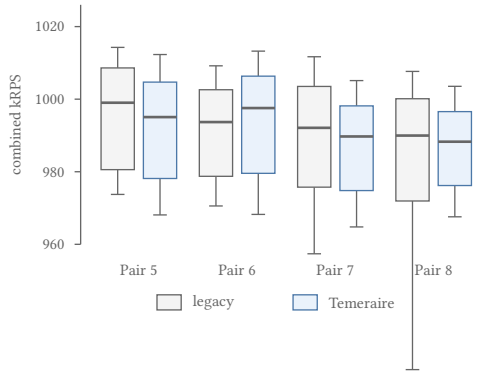

Figure 4: Per-trial combined throughput for the four release-off pairs of the correction run. Boxes span the interquartile range, whiskers the 5th to 95th percentile, and the heavy line the median of 2000 trials. The within-block spread is roughly 4%, an order of magnitude wider than the pairwise deltas in Table 2.

Release-on at 16 MiB/s reproduces the paper. All four pairs favor Temeraire and the mean is +0.401%, with an interval that excludes zero. Normalizing for CPU moves the mean to +0.474%, and that interval excludes zero as well. The paper's +0.44% falls inside both. This is the only condition in the study that separates from the baseline in Temeraire's favor, and it does so in the paper's own unit and within its reported value. Absolute throughput held between 980 and 995 kRPS across all sixteen blocks, a spread of 1.6%.

Release-off does not resolve. The mean is +0.022% and the median +0.037%, with an interval that covers zero and, once normalized, covers the paper's +0.75% as well. Figure 4 shows why four pairs are not enough here: the per-trial distributions inside each block span roughly 4%, an order of magnitude wider than the pairwise deltas that separate them, so the allocator difference is a small shift between two heavily overlapping populations.

Normalizing for CPU changes little, and the CPU record explains why. Sequential blocks spent about 4040 CPU seconds across roughly 4100 seconds of wall clock, and combined blocks 14 860 across 14 900, so the Redis process held between 98 and 99.7 percent of a single core throughout. Redis 6.0.9 executes commands on one thread and the benchmark keeps that thread saturated, which makes requests per second and requests per second per core nearly the same quantity for this workload. The metric gap identified in Section 4 is therefore real but small here, worth under a tenth of a percentage point. That conclusion is specific to a single-threaded saturated server and should not be carried over to the multithreaded applications in the paper's Table 1.

5.3 The second reading of the workload

The combined reading costs far more and measures far less. A block takes 4.11 hours against 1.14 for the sequential form, since an `EVAL` request carries Lua interpreter work that a plain command does not, and its throughput sits near 135 kRPS rather than 990. Its release-off median of +0.842% lands near the paper's +0.75%, but with an interval more than two percentage points wide that agreement

Table 3: Correction run on node85, combined EVAL workload, where one request performs the push and the read. Same interval construction as Table 2.

Pair	Order	Release off	Release on
1	L first	−0.260%	−1.121%
2	T first	+0.817%	−0.085%
3	L first	+1.287%	−0.304%
4	T first	+0.867%	+2.506%
Mean		+0.678%	+0.249%
Median		+0.842%	−0.194%
95% interval		[−0.37, +1.73]	[−2.21, +2.75]

carries little weight. Its release-on column is worse still: one pair at +2.506% drives a positive mean out of three negative pairs, and the interval spans five percentage points. CPU normalization shifts these figures by less than 0.01 percentage points, because these blocks are the most saturated of the study.

Running both readings was worth the compute even though one of them produced nothing usable. The choice the paper's sentence leaves open is not a detail: the two implementations differ by a factor of 3.6 in cost and by an order of magnitude in the width of the interval they yield. The sequential form is the better instrument, and Table 2 rests on it.

5.4 Release-rate sensitivity

The 16 MiB/s rate is a local parameter, since the paper never states one. The correction run measured only that rate, so the evidence on higher rates comes from the earlier series of July 2026, which used the same node, workload, and pair structure but recorded no CPU time. Its figures are therefore unnormalized, and Section 5.2 shows that for this workload the normalization would move them by under a tenth of a percentage point.

Table 4: Release-on sensitivity from the earlier series. Each cell compares Temeraire with the matched legacy block, and the four pairs alternate allocator order. Unnormalized throughput.

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Mean	Median
16 MiB/s	+1.319%	+0.149%	−0.358%	+0.942%	+0.513%	+0.545%
64 MiB/s	−0.631%	−0.094%	−0.674%	−0.574%	−0.494%	−0.603%
256 MiB/s	−0.028%	+0.402%	−0.133%	−1.419%	−0.294%	−0.080%

All four 64 MiB/s pairs favor legacy TCMalloc, and their interval, $[-0.92\%, -0.07\%]$, excludes zero. It is the only interval here that does so, which is worth recording but is weak evidence on its own: it rests on four pairs, on a normal approximation, and on one condition picked out of several tested without any correction for multiple comparisons. At 256 MiB/s three pairs are negative and one is positive; the −1.419% pair drives the negative mean, and inspection by trial quarter rules out a short collapse in that block, with Temeraire between 1.2% and 1.8% slower in each quarter. The sequence is not monotonic: performance changes sign between 16 and 64 MiB/s, then moves back toward zero at 256 MiB/s. Whatever the release rate does to this workload, it does not do it smoothly.

5.5 What the earlier series established

Two findings from the earlier work survive as evidence rather than as results.

The Docker/WSL2 stage produced a working and fully instrumented experiment whose measurements could not carry a sub-one-percent claim. Absolute throughput moved by more than 200 kRPS across a single series and later recovered. Each allocator block ran for about an hour, so a block inherited whatever the host happened to be doing at the time, and the largest apparent Temeraire improvements turned out to fall in the blocks that ran while the host was recovering from a slower period. That correlation is what makes the numbers uninterpretable rather than merely noisy: the allocator delta cannot be separated from the hour in which each block happened to run. Its release-on median reached +6.80% at 64 MiB/s, one 256 MiB/s pair reached +15.73%, and one release-on pair reached −12.02% and did not reproduce when rerun on its own. All three are an order of magnitude larger than the effect under study. A later audit then found that no run in that stage had performed periodic release at all: the runner started TCMalloc's background-actions thread without setting a release rate, and the pinned revision defaults that rate to zero, so the thread continued its per-CPU-cache work and calculated zero bytes for every pageheap release call. Those logs are retained as a record of the misconfiguration and excluded from every comparison.

The first bare-metal series, run in July on the same node, remains useful as a second sample of release-off. It gave a median of −0.34% where the correction run gives +0.037%, with intervals that overlap across almost their whole width. Two independent four-pair samples that disagree in sign while agreeing within their intervals are the clearest available statement that this condition needs more pairs, not a firmer conclusion. Its release-on pairs point the same way as the correction run, so seven of the eight release-on pairs measured on node85 favor Temeraire.

6 Discussion and Related Work

At the methodology level, the Redis experiment was reconstructed locally as far as the public artifact allowed: the allocator split had to be rebuilt from public code, preload had to be verified at runtime, the compiler path had to be documented, and THP activity had to be observed rather than assumed.

The move to bare metal narrows one question raised by WSL2. The double-digit release-on deltas did not recur on node85, where every pair stays within 1.5%, so they were environment- or time-dependent rather than a repeatable property of Temeraire. The rerun cannot go further than that: it changed CPU generation, kernel, operating system, execution environment, and time period together, so it isolates none of them as the cause.

Reproducing the paper's release-on value is not the same as confirming the paper's claim, and the difference is the release rate. Since the paper omits its rate, 16 MiB/s has no claim to being the setting behind +0.44%, and the 64 MiB/s aggregate runs the other way with an interval that also excludes zero. An effect that reverses between two rates the paper never distinguishes is a weaker thing than a +0.44% headline suggests, and the sensitivity in Table 4 is the part of this study a reader should weigh against it.

Two measurement gaps limited how far these comparisons could be pushed, and Section 5.2 measured the size of both. The paper normalizes Redis throughput for CPU and reports requests per second per core, while the local figure is an unnormalized request rate joined across two operations by a harmonic mean this report chose. That gap turns out to be worth under a tenth of a percentage point for this workload, because a single-threaded Redis under a saturating benchmark makes the two units almost the same quantity. The workload gap is larger: the paper's one-million-request trial admits at least two implementations, and measuring both showed that they differ by a factor of 3.6 in cost and by an order of magnitude in the width of the interval they produce. Neither gap is closed by argument, and both are now quantified rather than conceded.

Four pairs per condition also remain few for sub-one-percent effects. Allocator blocks run consecutively, so slow changes in frequency, temperature, or system activity can survive inside a pair. The recorded metadata omits CPU governor, turbo state, temperature, and per-block allocator mappings; preload was verified once before the run rather than inside every result directory.

Environmental fidelity also remains limited. The paper specifies the Redis version, benchmark shape, CPU family, LLVM commit, and optimization level, and leaves the Linux distribution, kernel, THP policy, and background release rate for Redis unstated. One specified property was not matched: the paper's Redis servers used Intel Skylake Xeon processors, while node85 carries a Xeon Gold 5318N, an Ice Lake generation part. TLB capacity and hugepage handling changed between those generations, so the hardware most relevant to a TLB-pressure result is the one piece of the paper's stated environment that this reproduction does not reproduce. Node85 does remove Docker and virtualization, and the exact LLVM commit was matched, but the public TCMalloc revision remains a historical approximation of Google's internal allocator.

Temeraire sits alongside other memory-management work on huge pages. SuperMalloc uses huge pages for very large allocations but does not make the general pageheap layout hugepage-aware [3]. MallocPool improves TLB behavior by serving objects from a contiguous preallocated region, but it is not a general-purpose pageheap replacement [2]. LLAMA uses lifetime prediction to group allocations that are likely to die together, whereas Temeraire uses online placement heuristics without profiling [5]. Kernel-level systems such as Ingens and HawkEye manage huge pages from the OS side and are complementary to allocator-side placement [4, 6].

7 Conclusion

This report covers the Redis case study alone. The original repository began as a generic allocator benchmark and had to be narrowed to that case study, moved to Redis 6.0.9, and shifted from glibc and gperftools comparisons to legacy TCMalloc versus Temeraire, with historical TCMalloc builds, wrapper libraries, preload checks, compiler metadata, THP controls, and memory instrumentation.

The WSL2 stage produced a working and instrumented experiment, but host drift made its release-on sweep unsuitable as final evidence. Repeating the workload on dedicated bare-metal Linux removed those swings, and a second corrected series, run after the metric mismatch came to light, settles the release-on question. Across four fresh pairs Temeraire gains +0.401% in raw throughput and +0.474% in requests per CPU second, with every pair positive and both intervals excluding zero while containing the paper's +0.44%. Seven of the eight release-on pairs measured on node85 favor Temeraire.

Release-off stays unresolved. Two independent four-pair samples put it at −0.34% and +0.037%, with intervals that overlap almost entirely, so the condition needs more pairs rather than a firmer conclusion. The earlier reading of the negative sample as inconsistent with the paper does not survive the second.

The reproduction therefore reaches the paper's release-on Redis result in the paper's own unit, and reaches nothing conclusive elsewhere. Two of its three known deviations are now measured rather than assumed: CPU normalization is worth under a tenth of a percentage point for a single-threaded saturated Redis, while the choice between two readings of the paper's trial sentence changes the cost by a factor of 3.6 and the interval width by an order of magnitude. The third, Skylake against Ice Lake, needs different hardware. Its other contribution is the path from an unsuitable starting artifact, through build failures, a misconfigured release rate, a signal that killed a run, and a parsing fault that silently zeroed a day of measurements, to a series whose raw data, settings, and repairs can all be audited. Google's fleet-scale claims and the production-era environment stay outside its reach.

References

[1] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. https://www.usenix.org/conference/osdi21/presentation/hunter

[2] Marcus Jägemar. 2018. Mallocpool: Improving Memory Performance Through Contiguously TLB Mapped Memory. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1127–1130. doi:10.1109/ETFA.2018.8502619

[3] Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management* (Portland, OR, USA) *(ISMM '15)*. Association for Computing Machinery, New York, NY, USA, 41–55. doi:10.1145/2754169.2754178

[4] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and efficient huge page management with ingens. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 705–721.

[5] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 541–556. doi:10.1145/3373376.3378525

[6] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 347–360. doi:10.1145/3297858.3304064